

The AI QA Interview Bible - Volume 1

THE AI QA INTERVIEW BIBLE

Volume 1

AI & LLM Fundamentals

60 Interview Questions with Answers, QA Perspectives, Examples, Pitfalls, and Follow-ups

Prepared for experienced QA professionals transitioning into AI Quality Engineering

Mohit Gupta

2026 Edition

Copyright and Use

This book is an original study and interview-preparation resource. It is intended for personal learning, professional development, and interview practice. Product names and framework names belong to their respective owners.

Important Note

The answers are designed to help candidates explain concepts clearly in interviews. They are not a substitute for hands-on practice, official product documentation, security review, legal advice, or domain-specific validation.

Dedication

Dedicated to every software quality professional who chooses to reinvent, learn, and lead in the era of Artificial Intelligence.

Preface

AI quality engineering requires two kinds of thinking at the same time: understanding how AI systems work and knowing how to challenge them like a tester. This volume builds the technical foundation needed before studying hallucination evaluation, RAG testing, agent validation, DeepEval, RAGAS, governance, and AI test architecture.

How to Use This Book

For each question, first practice the short interview answer. Then study the detailed explanation and connect it to the QA perspective. Use the follow-up questions for mock interviews. Avoid memorizing wording; aim to explain the idea in your own language and support it with a practical example.

Recommended Study Method

Study one chapter at a time. Create a small notebook or digital flashcard for the short answers. After each chapter, explain three topics without looking at the book. Finally, apply the concepts to one sample product such as an HR chatbot, banking copilot, support assistant, or RAG knowledge base.

Table of Contents

Chapter 1: AI and Generative AI Foundations

Chapter 2: Large Language Models

Chapter 3: Transformers and Attention

Chapter 4: Tokens and Tokenization

Chapter 5: Embeddings and Semantic Similarity

Chapter 6: Vector Databases

Chapter 7: Temperature, Top-K and Top-P

Chapter 8: Context Windows and Memory

Chapter 9: Fine-Tuning and Adaptation

Chapter 10: Prompt Engineering Basics

Quick Revision Checklist

Glossary

Selected References

Chapter 1

AI and Generative AI Foundations

How the concepts relate

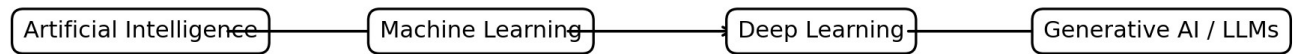


Figure: AI and Generative AI Foundations - conceptual overview

This chapter contains 6 interview questions. Focus on the short answer first, then connect the concept to practical QA decisions and test design.

Q1. What is Artificial Intelligence?

Interview-ready answer

Artificial Intelligence is the field of building systems that perform tasks normally requiring human intelligence, such as perception, reasoning, language understanding, prediction, and decision-making.

Detailed explanation

AI is an umbrella term rather than one specific technology. It includes rule-based systems, machine learning models, deep neural networks, computer vision, natural language processing, and generative models. A system can be called AI even when it does not learn continuously; what matters is that it performs an intelligent task using rules, learned patterns, or both. Modern enterprise AI often combines deterministic software with probabilistic models. This distinction matters because deterministic components can be tested with exact expected results, while probabilistic components need statistical, semantic, and risk-based evaluation.

QA / AI quality perspective

As a QA professional, first identify which components are deterministic and which are probabilistic. Define exact assertions for APIs, workflows, permissions, and data handling, while defining quality thresholds for model outputs such as relevance, groundedness, safety, and consistency.

Real-world example

A banking chatbot may use deterministic authentication and account APIs but an LLM to interpret a customer request and compose an answer.

Common mistakes to avoid

- Treating every automation rule as AI
- Assuming AI always means machine learning
- Testing probabilistic output with only exact string comparison

Likely follow-up questions

- How is AI different from automation?
- What is narrow AI?
- Why is AI testing different from traditional testing?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q2. How are AI, Machine Learning, Deep Learning, and Generative AI related?

Interview-ready answer

AI is the broad field; Machine Learning is a subset that learns patterns from data; Deep Learning is a subset of ML using multi-layer neural networks; Generative AI creates new content such as text, images, audio, or code.

Detailed explanation

These terms describe nested but overlapping capabilities. Traditional AI may use explicit rules. Machine learning estimates a function from examples. Deep learning uses representation learning to discover complex features. Generative AI learns the distribution or structure of data and produces new samples. Most modern LLMs are deep-learning-based generative models trained on large text corpora, then adapted through instruction tuning and alignment.

QA / AI quality perspective

The classification guides the test strategy. A rule engine requires decision-table coverage; a classifier requires accuracy and class-wise metrics; an LLM requires response-quality, safety, robustness, and non-determinism testing.

Real-world example

An email spam rule may block a fixed domain; an ML spam classifier predicts spam probability; a generative assistant writes a suggested reply.

Common mistakes to avoid

- Using AI and GenAI as synonyms
- Ignoring model-specific metrics
- Assuming the newest model is always the best for a use case

Likely follow-up questions

- What is supervised learning?
- What is a foundation model?
- Where does RAG fit?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q3. What is Generative AI?

Interview-ready answer

Generative AI refers to models that create new content by learning patterns in existing data and generating likely continuations or samples.

Detailed explanation

Unlike discriminative models that mainly classify or predict labels, generative systems produce text, images, code, speech, video, or structured data. A text LLM estimates the probability of the next token given prior tokens. The output can be fluent without being factually correct, so linguistic quality must not be confused with truth. Enterprise systems add retrieval, tools, guardrails, and workflow controls to make GenAI more useful and reliable.

QA / AI quality perspective

Test both the generated content and the surrounding system. Evaluate correctness, relevance, completeness, groundedness, safety, bias, format compliance, latency, cost, and failure handling.

Real-world example

A support copilot retrieves product documentation and drafts a troubleshooting answer for an agent.

Common mistakes to avoid

- Assuming fluent output is correct
- Evaluating only happy-path prompts
- Ignoring prompt injection and data leakage

Likely follow-up questions

- What are common GenAI risks?
- How do LLMs generate text?
- What is hallucination?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q4. What is the difference between predictive AI and generative AI?

Interview-ready answer

Predictive AI estimates a label, score, or future outcome, while generative AI produces new content or sequences.

Detailed explanation

Predictive systems answer questions such as “Will this transaction be fraudulent?” or “What will demand be next week?” Generative systems answer questions such as “Explain this transaction” or “Draft a demand forecast summary.” Many products combine both: a predictive model supplies a score and an LLM explains it. The two categories have different evaluation needs. Predictive models use confusion matrices, precision, recall, calibration, and drift monitoring. Generative systems additionally require semantic quality, grounding, safety, and consistency evaluation.

QA / AI quality perspective

QA should verify the boundary between prediction and generation. Confirm that generated explanations do not contradict the underlying score and that uncertainty is communicated correctly.

Real-world example

A fraud model predicts 0.92 risk; an LLM produces a human-readable explanation based on the model features and policy rules.

Common mistakes to avoid

- Applying only accuracy to an LLM
- Allowing the LLM to invent reasons for a prediction
- Ignoring calibration and uncertainty

Likely follow-up questions

- Can a product use both types?
- What is explainability?
- How do you test generated explanations?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q5. What is a foundation model?

Interview-ready answer

A foundation model is a large model trained on broad data that can be adapted to many downstream tasks through prompting, retrieval, fine-tuning, or tools.

Detailed explanation

Foundation models are reusable general-purpose capabilities rather than task-specific models. Their broad training enables zero-shot and few-shot behavior, but also introduces uncertainty about training data, bias, and domain suitability. Adaptation methods range from prompt design and RAG to parameter-efficient fine-tuning. The model is only one component of a production AI system; quality also depends on data, orchestration, retrieval, guardrails, and monitoring.

QA / AI quality perspective

Test the model-in-system, not only the base model. Record model name and version, prompt version, decoding parameters, retrieved context, and tool results so failures are reproducible.

Real-world example

A company uses one foundation model for summarization, support search, code assistance, and document extraction, each with different prompts and controls.

Common mistakes to avoid

- Treating a foundation model as a complete product
- Not tracking model version changes
- Assuming vendor benchmarks predict your domain performance

Likely follow-up questions

- What is model adaptation?
- What is instruction tuning?
- Why do models need guardrails?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q6. Why is AI testing different from traditional software testing?

Interview-ready answer

AI outputs are often probabilistic, context-sensitive, data-dependent, and difficult to validate with a single exact expected result.

Detailed explanation

Traditional software is mostly deterministic: identical inputs under identical conditions should produce identical outputs. AI systems may produce multiple acceptable answers, fail silently, degrade when data changes, or behave differently across model versions. Therefore, testing requires datasets, rubrics, thresholds, statistical comparisons, human review, adversarial inputs, and continuous monitoring. Traditional functional, API, security, performance, and integration testing still remain essential.

QA / AI quality perspective

Use layered testing: deterministic checks for infrastructure and contracts; semantic evaluators for model outputs; human evaluation for subjective or high-risk cases; production monitoring for drift and emerging failures.

Real-world example

A chatbot response can be differently worded yet correct. Exact text comparison would fail a good answer and might pass a copied but contextually wrong answer.

Common mistakes to avoid

- Discarding traditional testing
- Using one prompt as evidence of quality
- Expecting 100% consistency from a stochastic model

Likely follow-up questions

- What are golden datasets?
- How can quality thresholds be defined?
- What is human-in-the-loop evaluation?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Chapter 2

Large Language Models

Simplified LLM inference flow

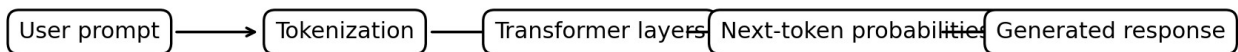


Figure: Large Language Models - conceptual overview

This chapter contains 6 interview questions. Focus on the short answer first, then connect the concept to practical QA decisions and test design.

Q7. What is a Large Language Model (LLM)?

Interview-ready answer

An LLM is a neural language model trained on very large text datasets to predict and generate token sequences, enabling tasks such as answering, summarizing, translating, and coding.

Detailed explanation

An LLM learns statistical relationships among tokens and concepts. During inference, it converts input tokens into internal representations, processes them through transformer layers, and predicts a probability distribution for the next token. Repeating this step generates a response. LLMs do not retrieve truth from a database by default; they generate likely text based on learned patterns and current context.

QA / AI quality perspective

Validate both capability and limitations. Design tests for domain knowledge, instruction following, unsupported claims, ambiguity, multilingual behavior, long context, safety, and consistency.

Real-world example

Given a defect description, an LLM can generate a concise summary and suggested severity, but QA must verify that the severity follows actual business rules.

Common mistakes to avoid

- Describing an LLM as a database
- Assuming it understands exactly like a human
- Ignoring token and context limits

Likely follow-up questions

- How does next-token prediction work?
- Why do LLMs hallucinate?
- What is an instruction-tuned model?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q8. How does an LLM generate a response?

Interview-ready answer

It tokenizes the prompt, processes token representations through transformer layers, predicts next-token probabilities, selects a token using decoding settings, and repeats until stopping.

Detailed explanation

The model begins with the conversation context. Each decoding step produces logits, which are converted into probabilities. Temperature and filters such as top-k and top-p influence which candidate token is selected. The selected token is appended to the context and the process repeats. Generation ends when a stop token, stop sequence, or token limit is reached.

QA / AI quality perspective

Capture the full request configuration in test evidence: system prompt, user prompt, conversation history, model, temperature, top-p, max output tokens, tools, and retrieved context. Otherwise, defects may be difficult to reproduce.

Real-world example

A response such as “Reset the router” is produced token by token, not retrieved as one stored sentence.

Common mistakes to avoid

- Thinking the whole answer appears at once
- Not logging model parameters
- Ignoring stop conditions and truncation

Likely follow-up questions

- What are logits?
- What is deterministic decoding?
- What causes repeated text?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q9. What is pretraining in an LLM?

Interview-ready answer

Pretraining is the large-scale learning phase in which a model learns language patterns and general knowledge, commonly through next-token prediction.

Detailed explanation

During pretraining, the model processes massive corpora and adjusts parameters to reduce prediction error. It learns grammar, style, associations, factual patterns, and some reasoning behavior. Pretraining is computationally expensive and usually performed by model providers. It does not guarantee instruction following or safe behavior, so post-training stages are added.

QA / AI quality perspective

QA teams generally do not test pretraining directly, but they should understand its consequences: unknown data provenance, stale knowledge, memorization risk, bias, and uneven domain coverage.

Real-world example

A pretrained model may complete text well but fail to follow a structured JSON instruction until it is instruction-tuned.

Common mistakes to avoid

- Confusing pretraining with fine-tuning
- Assuming all learned knowledge is current
- Ignoring memorization and privacy risk

Likely follow-up questions

- What is post-training?
- What is instruction tuning?
- What is RLHF?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q10. What is instruction tuning?

Interview-ready answer

Instruction tuning adapts a pretrained model using examples of instructions and desired responses so it follows user tasks more reliably.

Detailed explanation

Instruction datasets contain pairs or conversations showing how to answer, refuse, format, reason, or use tools. The model learns behavioral patterns beyond plain text completion. Instruction tuning improves usability but does not eliminate hallucination or safety issues. Different providers use different mixtures of supervised fine-tuning, preference optimization, and safety alignment.

QA / AI quality perspective

Test whether the model follows system priorities, output schemas, refusal rules, and conflicting instructions. Include adversarial prompts that attempt to override the intended behavior.

Real-world example

A base model may continue “Translate English to French:” as text, whereas an instruction-tuned model returns only the translation.

Common mistakes to avoid

- Believing instruction tuning guarantees obedience
- Testing only friendly prompts
- Ignoring system-user instruction hierarchy

Likely follow-up questions

- What is alignment?
- What is prompt injection?
- How is instruction tuning different from prompting?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q11. What is alignment in LLMs?

Interview-ready answer

Alignment is the process of shaping model behavior so it follows human intentions, policies, and safety expectations.

Detailed explanation

Alignment can involve supervised examples, human or AI preferences, reward models, policy optimization, constitutional rules, and safety filters. It is not a single solved property. A model can be helpful in one domain yet unsafe, biased, or overly refusing in another. Alignment decisions also involve trade-offs among helpfulness, harmlessness, honesty, and user freedom.

QA / AI quality perspective

Create policy-based test suites for allowed, disallowed, and borderline requests. Measure both unsafe compliance and excessive refusal. Validate consistent behavior across paraphrases and languages.

Real-world example

A medical assistant should refuse diagnosis beyond scope while still offering safe general guidance and escalation advice.

Common mistakes to avoid

- Treating alignment as perfect
- Checking only unsafe compliance
- Ignoring over-refusal and false positives

Likely follow-up questions

- What is RLHF?
- How do guardrails differ from alignment?
- What is a safety benchmark?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q12. Why do LLMs hallucinate?

Interview-ready answer

LLMs hallucinate because they optimize for likely text generation, not guaranteed factual truth, and may lack reliable context, current knowledge, or uncertainty calibration.

Detailed explanation

Causes include ambiguous prompts, missing knowledge, conflicting context, poor retrieval, long-context distraction, training-data errors, and decoding randomness. Hallucination can be factual, contextual, citation-based, or reasoning-related. Mitigation includes better prompts, RAG, tool use, constrained outputs, verification, abstention rules, and human review.

QA / AI quality perspective

Test hallucination with unanswerable questions, conflicting evidence, outdated facts, citation checks, and source-grounded datasets. Measure unsupported claim rate rather than relying only on subjective impressions.

Real-world example

An internal HR assistant invents a leave policy when the relevant document is absent from the knowledge base.

Common mistakes to avoid

- Equating every wrong answer with hallucination
- Trying to solve hallucination only by lowering temperature
- Not distinguishing retrieval failure from generation failure

Likely follow-up questions

- How do you detect hallucination?
- What is faithfulness?
- How does RAG reduce hallucination?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Chapter 3

Transformers and Attention

Transformer block - simplified



Figure: Transformers and Attention - conceptual overview

This chapter contains 6 interview questions. Focus on the short answer first, then connect the concept to practical QA decisions and test design.

Q13. What is the Transformer architecture?

Interview-ready answer

The Transformer is a neural architecture that processes sequences primarily through attention mechanisms, enabling parallel training and strong long-range context modeling.

Detailed explanation

Transformers replace recurrence with self-attention and feed-forward layers. Token embeddings are combined with positional information. Multiple attention heads learn different relationships among tokens. Residual connections and normalization stabilize training. Decoder-only transformers are common for text generation, while encoder-only and encoder-decoder variants suit understanding and sequence-to-sequence tasks.

QA / AI quality perspective

You do not test internal layers directly in most product QA. Instead, design behavioral tests derived from known limitations: positional sensitivity, long-context degradation, instruction conflict, and attention to irrelevant text.

Real-world example

In “The server failed because it exhausted memory,” attention helps associate “it” with “server.”

Common mistakes to avoid

- Saying transformers understand grammar through fixed rules
- Assuming attention always finds the correct relation
- Overloading interviews with unnecessary mathematics

Likely follow-up questions

- What is self-attention?
- What are encoder and decoder models?
- Why are transformers scalable?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q14. What is self-attention?

Interview-ready answer

Self-attention lets each token weigh the relevance of other tokens in the same sequence when building its contextual representation.

Detailed explanation

For each token, the model forms query, key, and value vectors. Similarity between a query and keys produces attention weights, which combine the values. This allows a word to be interpreted using distant context. The process is repeated across multiple heads and layers. Attention is powerful but not equivalent to verified reasoning or factual grounding.

QA / AI quality perspective

Test pronoun resolution, conflicting statements, distractors, reordered context, and long documents to see whether the model attends to the relevant evidence.

Real-world example

In a policy document, the model must connect “this exception” to the correct preceding eligibility rule, not a nearby unrelated rule.

Common mistakes to avoid

- Calling attention a database search
- Assuming high attention means causation
- Ignoring distractor sensitivity

Likely follow-up questions

- What are query, key, and value?
- What is multi-head attention?
- How is attention tested behaviorally?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q15. What is multi-head attention?

Interview-ready answer

Multi-head attention runs several attention operations in parallel so the model can capture different relationships and representation subspaces.

Detailed explanation

One head may focus on syntax, another on entity relationships, and another on positional patterns, although individual heads are not guaranteed to have clean human-interpretable roles. Their outputs are concatenated and transformed. This improves representation capacity compared with one attention pattern.

QA / AI quality perspective

QA should avoid claiming specific internal-head behavior without interpretability evidence. Focus on output-level tests for relationships, instructions, and context usage.

Real-world example

A model answering a support query may simultaneously track product name, error code, and troubleshooting sequence.

Common mistakes to avoid

- Assuming each head has a fixed named job
- Using attention visualization as proof of correctness
- Ignoring end-to-end validation

Likely follow-up questions

- Can attention be explained?
- What is an attention head?
- Why do models need many layers?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q16. What is positional encoding?

Interview-ready answer

Positional encoding provides token-order information because self-attention alone does not inherently represent sequence order.

Detailed explanation

Position information can be added through fixed sinusoidal encodings, learned embeddings, or relative/rotary methods. It helps distinguish sequences with the same words in different orders. Position handling also influences how models behave near context limits and on sequences longer than those used in training.

QA / AI quality perspective

Test order-sensitive instructions, numbered steps, chronology, and repeated content. Long-context evaluations should vary where critical evidence appears.

Real-world example

“User approved manager” and “Manager approved user” contain similar tokens but different order and meaning.

Common mistakes to avoid

- Saying positional encoding stores absolute meaning
- Ignoring order-sensitive test cases
- Assuming all positions are handled equally well

Likely follow-up questions

- What is RoPE?
- Why can middle context be missed?
- How do positions affect long prompts?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q17. What is an encoder-only, decoder-only, and encoder-decoder model?

Interview-ready answer

Encoder-only models produce contextual representations, decoder-only models generate sequences, and encoder-decoder models map an input sequence to an output sequence.

Detailed explanation

Encoder-only architectures are commonly used for classification and embeddings. Decoder-only architectures predict the next token and power many conversational LLMs. Encoder-decoder architectures encode the source and generate a target, often used for translation and summarization. Product selection should depend on task, latency, cost, and quality needs.

QA / AI quality perspective

Test the actual model type according to its role. An embedding model needs retrieval-quality tests; a generative model needs response evaluation; a classifier needs label metrics.

Real-world example

A support system may use an encoder to create embeddings and a decoder LLM to generate the final answer.

Common mistakes to avoid

- Calling all transformers LLM chatbots
- Using generation metrics for embeddings
- Ignoring multi-model pipeline failures

Likely follow-up questions

- Which architecture is used for embeddings?
- What is a cross-encoder?
- Why combine models?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q18. What are model parameters?

Interview-ready answer

Parameters are learned numerical weights that determine how a neural network transforms inputs into outputs.

Detailed explanation

During training, optimization adjusts billions of weights to reduce loss. Parameter count is one indicator of capacity but does not directly guarantee quality. Data quality, architecture, training method, context length, inference optimization, and alignment also matter. Smaller specialized models can outperform larger models for a narrow task.

QA / AI quality perspective

Do not select or approve a model only by parameter count. Benchmark representative tasks, safety, latency, cost, and operational constraints.

Real-world example

A compact domain model may classify support tickets more accurately and cheaply than a larger general LLM.

Common mistakes to avoid

- Equating more parameters with better production quality
- Ignoring latency and cost
- Comparing models on vendor claims only

Likely follow-up questions

- What is model capacity?
- What is quantization?
- How should QA compare models?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Chapter 4

Tokens and Tokenization

Simplified LLM inference flow

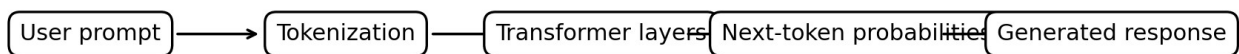


Figure: Tokens and Tokenization - conceptual overview

This chapter contains 6 interview questions. Focus on the short answer first, then connect the concept to practical QA decisions and test design.

Q19. What is a token?

Interview-ready answer

A token is a unit of text processed by a language model, such as a word, subword, punctuation mark, or character sequence.

Detailed explanation

Models do not directly read words. A tokenizer maps text to token IDs. Common words may be one token, while rare words are split. Token count affects context usage, output limits, latency, and cost. Token boundaries vary by model and language, so character count is not a reliable substitute.

QA / AI quality perspective

Create tests for long inputs, multilingual content, code, special characters, and near-limit prompts. Track token usage in performance and cost reports.

Real-world example

“unbelievable” might be represented as one token in one tokenizer and several subword tokens in another.

Common mistakes to avoid

- Assuming one word equals one token
- Ignoring hidden system and tool tokens
- Using character limits as token limits

Likely follow-up questions

- How are tokens created?
- Why does tokenization affect cost?
- What is a token ID?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q20. What is tokenization?

Interview-ready answer

Tokenization is the process of splitting text into model-specific tokens and converting them into numeric IDs.

Detailed explanation

Tokenizers are usually trained to balance vocabulary size and sequence length. Subword methods handle unknown words by composing smaller units. The same sentence can produce different token counts across models. Tokenization can also expose edge cases involving whitespace, Unicode, emojis, mixed scripts, and prompt boundaries.

QA / AI quality perspective

Test Unicode normalization, invisible characters, repeated spaces, emojis, markup, and multilingual prompts. Security testing should consider tokenization-based prompt bypasses.

Real-world example

A malicious instruction split with unusual Unicode may evade a naive keyword filter but still be interpreted by the model.

Common mistakes to avoid

- Testing only English text
- Ignoring normalization
- Assuming filters and models tokenize identically

Likely follow-up questions

- What is subword tokenization?
- What are tokenization attacks?
- How do emojis tokenize?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q21. Why do LLM APIs charge by tokens?

Interview-ready answer

Token-based pricing reflects the computational work required to process input context and generate output tokens.

Detailed explanation

Input and output may have different prices because generation is often more expensive than prompt processing. Long histories, retrieved documents, tool results, and verbose outputs increase cost. Token budgets also affect latency and context availability. Production systems therefore use truncation, summarization, caching, model routing, and output limits.

QA / AI quality perspective

Include cost as a quality attribute. Measure average and percentile token usage by scenario, detect regressions, and define budgets or quality gates.

Real-world example

A prompt update that adds a 20-page policy document to every request may improve accuracy but multiply latency and cost.

Common mistakes to avoid

- Monitoring only output tokens
- Ignoring tool and retrieval context
- Optimizing cost without checking quality impact

Likely follow-up questions

- How can token cost be reduced?
- What is semantic caching?
- What is model routing?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q22. What is a vocabulary in an LLM?

Interview-ready answer

A vocabulary is the finite set of tokens that a tokenizer can map directly to IDs.

Detailed explanation

Vocabulary design affects compression, multilingual coverage, and handling of rare terms. A larger vocabulary can reduce sequence length but increases embedding and output dimensions. Domain terms may still be split into subwords. The vocabulary is fixed for a deployed tokenizer even though the model can combine tokens to represent new words.

QA / AI quality perspective

Test domain terms, product codes, names, and languages that may tokenize inefficiently or ambiguously. Retrieval and classification quality can degrade if important entities are fragmented.

Real-world example

A medical drug name may split into many tokens, increasing context usage and risking spelling errors in generation.

Common mistakes to avoid

- Assuming unknown words cannot be processed
- Treating vocabulary as a dictionary of meanings
- Ignoring domain identifiers

Likely follow-up questions

- Can new words be represented?
- How does vocabulary affect multilingual models?
- What is an out-of-vocabulary token?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q23. How does tokenization affect multilingual applications?

Interview-ready answer

Languages differ in how efficiently their text is represented, so token counts, cost, latency, and quality can vary across languages.

Detailed explanation

A tokenizer optimized heavily for English may split another script into more tokens. Longer token sequences consume context and can reduce effective capacity. Translation, code-mixing, transliteration, and locale-specific punctuation also affect behavior.

QA / AI quality perspective

Run language-wise benchmarks rather than assuming English results generalize. Compare token counts, response quality, refusal behavior, and latency for each supported locale.

Real-world example

A Hindi-English mixed customer query may use more tokens and produce less consistent entity extraction than an equivalent English query.

Common mistakes to avoid

- Using translated English tests only
- Ignoring token budget differences
- Assuming multilingual means equal quality

Likely follow-up questions

- How do you test code-mixed prompts?
- What is locale robustness?
- How can multilingual cost be measured?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q24. What is a maximum output token limit?

Interview-ready answer

It is the configured upper bound on how many tokens a model may generate for a response.

Detailed explanation

The limit prevents uncontrolled cost and latency but can truncate content. Some APIs share a total context budget between input and output, while others expose separate controls. Stop sequences can end generation earlier. The desired value depends on task and output schema.

QA / AI quality perspective

Test boundary behavior, truncation detection, incomplete JSON, missing conclusions, and retry or continuation logic. A system should not silently present a cut-off answer as complete.

Real-world example

A report-generation request hits the token limit and returns a JSON object without a closing brace.

Common mistakes to avoid

- Setting a high limit without cost controls
- Not validating completion reason
- Assuming truncation is a model-quality issue only

Likely follow-up questions

- What is a stop sequence?
- How do you handle truncated JSON?
- How does context budget interact with output length?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Chapter 5

Embeddings and Semantic Similarity

Embedding and vector retrieval workflow

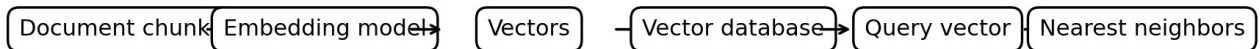


Figure: Embeddings and Semantic Similarity - conceptual overview

This chapter contains 6 interview questions. Focus on the short answer first, then connect the concept to practical QA decisions and test design.

Q25. What is an embedding?

Interview-ready answer

An embedding is a dense numeric vector that represents the semantic characteristics of text, images, or other data.

Detailed explanation

Embedding models map similar content to nearby points in a vector space. They enable semantic search, clustering, recommendation, deduplication, and retrieval for RAG. Embeddings are not human-readable and their meaning depends on the model. Changing the embedding model usually requires re-embedding the stored corpus.

QA / AI quality perspective

Test retrieval quality with labeled query-document pairs, hard negatives, domain terminology, multilingual queries, and model-version changes. Monitor both relevance and latency.

Real-world example

“Reset my password” and “I cannot access my account” may have similar vectors despite sharing few exact words.

Common mistakes to avoid

- Calling embeddings keywords
- Comparing vectors from different models directly
- Assuming semantic similarity equals business relevance

Likely follow-up questions

- How are embeddings generated?
- What is cosine similarity?
- Why must indexes be rebuilt after model changes?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q26. What is semantic similarity?

Interview-ready answer

Semantic similarity measures how close two items are in meaning rather than exact wording.

Detailed explanation

It is often estimated by comparing embeddings using cosine similarity, dot product, or distance. High similarity does not guarantee factual equivalence or task relevance. Thresholds depend on the model and dataset and should be calibrated empirically.

QA / AI quality perspective

Create positive pairs, near-miss pairs, and misleading hard negatives. Evaluate ranking metrics rather than relying on one arbitrary similarity threshold.

Real-world example

“Cancel my subscription” is semantically close to “terminate my plan” but should not match “pause my subscription” if the business actions differ.

Common mistakes to avoid

- Using lexical overlap as semantic truth
- Choosing thresholds without data
- Ignoring action-level differences

Likely follow-up questions

- What is a hard negative?
- How do you choose a similarity threshold?
- What is ranking quality?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q27. What is cosine similarity?

Interview-ready answer

Cosine similarity compares the angle between two vectors, focusing on direction rather than magnitude.

Detailed explanation

Its value is commonly between -1 and 1, though practical embedding distributions vary. A higher value often means greater semantic similarity. Some vector databases use cosine distance, which reverses the interpretation, so metric definitions must be verified.

QA / AI quality perspective

Confirm the distance metric configured in the index and API. Test threshold and ranking behavior with known pairs, including duplicates and unrelated items.

Real-world example

Two support-ticket embeddings with cosine similarity 0.90 may be more related than a pair at 0.55, but the acceptable threshold must come from validation data.

Common mistakes to avoid

- Confusing distance with similarity
- Copying thresholds across models
- Assuming high similarity proves correctness

Likely follow-up questions

- What is dot product?
- What is Euclidean distance?
- Which metric should be used?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q28. What is an embedding model?

Interview-ready answer

An embedding model is a model optimized to convert input data into vectors suitable for similarity-based tasks.

Detailed explanation

Embedding models differ in dimensions, maximum input length, domain performance, multilingual support, latency, cost, and whether query and document inputs require different instructions. Bi-encoders create independent vectors efficiently; cross-encoders jointly score pairs more accurately but more slowly.

QA / AI quality perspective

Benchmark candidate models using your corpus and queries. Include retrieval metrics, resource usage, and failure cases. Do not rely only on public leaderboards.

Real-world example

A legal-search application may choose a domain embedding model that outperforms a general model on clauses and citations.

Common mistakes to avoid

- Using the generative LLM itself as the only embedding strategy
- Ignoring input truncation
- Selecting by dimension count alone

Likely follow-up questions

- What is a bi-encoder?
- What is a cross-encoder reranker?
- How do you compare embedding models?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q29. What is dimensionality in embeddings?

Interview-ready answer

Dimensionality is the number of numeric components in each embedding vector.

Detailed explanation

Higher dimension can represent more nuanced patterns but increases storage, memory, and search cost. More dimensions do not automatically mean better retrieval. Some services support dimension reduction, which must be benchmarked for quality impact.

QA / AI quality perspective

Test retrieval before and after dimension changes and assess storage, latency, and index size. Treat dimension configuration as a versioned component.

Real-world example

Reducing vectors from 3,072 to 1,024 dimensions may lower cost while preserving acceptable recall for a specific corpus.

Common mistakes to avoid

- Assuming larger dimensions are always superior
- Changing dimensions without rebuilding indexes
- Ignoring infrastructure impact

Likely follow-up questions

- What is vector compression?
- What is quantization?
- How does dimension affect latency?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q30. What are common embedding failure modes?

Interview-ready answer

Common failures include weak domain representation, truncation, multilingual gaps, entity confusion, hard-negative similarity, stale indexes, and model-version mismatch.

Detailed explanation

Embeddings compress meaning into vectors, so fine distinctions can be lost. Similar templates can dominate content differences. Product codes, negation, numbers, and dates may be poorly represented. An index can also contain missing, duplicated, or outdated vectors.

QA / AI quality perspective

Build targeted test sets for negation, numeric differences, near-duplicate documents, rare entities, and stale content. Validate ingestion counts and metadata as well as semantic ranking.

Real-world example

“Account is active” and “Account is not active” may be dangerously close if the model underweights negation.

Common mistakes to avoid

- Testing only obvious synonyms
- Ignoring ingestion integrity
- Blaming generation for retrieval errors

Likely follow-up questions

- How do hard negatives help?
- How do you test stale vectors?
- What is retrieval recall?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Chapter 6

Vector Databases

Embedding and vector retrieval workflow

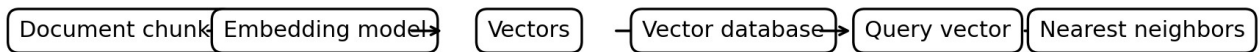


Figure: Vector Databases - conceptual overview

This chapter contains 6 interview questions. Focus on the short answer first, then connect the concept to practical QA decisions and test design.

Q31. What is a vector database?

Interview-ready answer

A vector database stores and indexes embeddings so applications can efficiently find semantically similar items.

Detailed explanation

It combines vector search with metadata, filtering, persistence, scaling, and operational APIs. Approximate nearest-neighbor algorithms make large-scale search fast by trading a small amount of exactness for performance. Examples include managed and self-hosted systems, but product choice should be based on scale, latency, filtering, security, and operations.

QA / AI quality perspective

Test indexing, upsert, delete, metadata filters, tenancy isolation, retrieval relevance, consistency, backup, and performance at expected scale.

Real-world example

A RAG assistant searches millions of document chunks by embedding similarity and filters results by user department and document status.

Common mistakes to avoid

- Treating a vector database as only a file store
- Ignoring access control
- Testing semantic relevance without metadata filters

Likely follow-up questions

- What is ANN search?
- How is a vector database different from a vector index?
- What should be performance tested?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q32. How does vector search work?

Interview-ready answer

The query is embedded into the same vector space as stored items, and the system returns the nearest vectors according to a similarity or distance metric.

Detailed explanation

Exact search compares against every vector and can be expensive. Approximate indexes such as HNSW or IVF reduce search time. Results may be filtered or reranked. Search quality depends on the embedding model, chunking, index parameters, filters, and query formulation.

QA / AI quality perspective

Separate retrieval testing from generation testing. Measure recall@k, precision@k, MRR, and latency using a labeled dataset. Vary k and index settings.

Real-world example

A query vector retrieves the top five support-ticket chunks, then a reranker promotes the most directly relevant two.

Common mistakes to avoid

- Measuring only latency
- Assuming nearest means correct
- Ignoring index approximation errors

Likely follow-up questions

- What is HNSW?
- What is top-k retrieval?
- Why use reranking?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q33. What is Approximate Nearest Neighbor (ANN) search?

Interview-ready answer

ANN search finds vectors that are probably among the nearest neighbors without exhaustively comparing every stored vector.

Detailed explanation

ANN algorithms build structures that navigate the vector space efficiently. They expose tuning trade-offs among recall, latency, memory, and build time. HNSW uses a layered proximity graph; other approaches use clusters or quantization.

QA / AI quality perspective

Benchmark at production scale and report both retrieval recall and latency percentiles. An extremely fast index with poor recall can cause downstream hallucination or irrelevant answers.

Real-world example

An ANN index returns the right policy chunk in 98% of test queries at p95 latency below the service target.

Common mistakes to avoid

- Calling approximate search inaccurate by definition
- Optimizing only average latency
- Not testing after index rebuilds

Likely follow-up questions

- What is HNSW?
- How do you tune ANN recall?
- What is p95 latency?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q34. What is metadata filtering in vector search?

Interview-ready answer

Metadata filtering restricts candidate vectors using structured attributes such as tenant, language, date, access level, or document type.

Detailed explanation

Filtering is critical for relevance and security. Depending on the database, filters may be applied before, during, or after vector search. Weak filter implementation can reduce recall or leak unauthorized content.

QA / AI quality perspective

Test positive and negative authorization cases, multi-tenant isolation, filter combinations, missing metadata, stale permissions, and performance under selective filters.

Real-world example

A user in Finance must retrieve only approved Finance policies even when an HR document has a more similar vector.

Common mistakes to avoid

- Treating filters as optional relevance features
- Not testing unauthorized nearest neighbors
- Ignoring null or inconsistent metadata

Likely follow-up questions

- How do filters affect recall?
- What is tenant isolation?
- How should access control be validated?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q35. What is chunking, and why does it matter for vector databases?

Interview-ready answer

Chunking divides source documents into retrieval units that can be embedded and searched.

Detailed explanation

Chunks that are too large dilute relevance and consume context; chunks that are too small lose meaning and relationships. Strategies include fixed size, recursive, semantic, sentence, section, and structure-aware chunking, often with overlap. Chunk metadata should preserve source, page, heading, and permissions.

QA / AI quality perspective

Compare chunking variants on the same labeled query set. Evaluate retrieval metrics, answer faithfulness, citation quality, cost, and latency. Include tables, lists, headings, and cross-section questions.

Real-world example

A leave policy should keep eligibility and exception clauses together rather than splitting them across unrelated chunks.

Common mistakes to avoid

- Using one chunk size for all document types
- Ignoring document structure
- Evaluating generation without checking retrieved chunks

Likely follow-up questions

- What is chunk overlap?
- What is semantic chunking?
- How do you select chunk size?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q36. How do you test a vector database?

Interview-ready answer

Test functional CRUD operations, retrieval quality, metadata behavior, security, consistency, scale, latency, resilience, and index lifecycle.

Detailed explanation

A complete strategy covers ingestion accuracy, duplicate handling, dimension validation, index creation, upsert semantics, deletion, filters, nearest-neighbor ranking, concurrency, backup/restore, failover, and observability. Quality testing requires labeled queries; platform testing requires load and fault scenarios.

QA / AI quality perspective

Create separate suites for data pipeline, semantic relevance, authorization, performance, and resilience. Track corpus version and embedding version in every run.

Real-world example

After deleting an expired policy, verify that neither the vector nor cached retrieval results can return it.

Common mistakes to avoid

- Running only API smoke tests
- Not validating source-to-index counts
- Ignoring stale cache and eventual consistency

Likely follow-up questions

- What KPIs should be tracked?
- How do you test reindexing?
- What defects occur in ingestion?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Chapter 7

Temperature, Top-K and Top-P

How decoding controls shape an answer

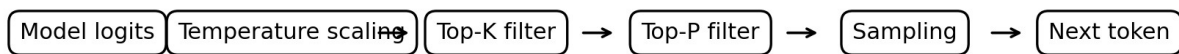


Figure: Temperature, Top-K and Top-P - conceptual overview

This chapter contains 6 interview questions. Focus on the short answer first, then connect the concept to practical QA decisions and test design.

Q37. What is temperature in an LLM?

Interview-ready answer

Temperature controls how strongly the model favors high-probability tokens; lower values make output more predictable, while higher values increase diversity.

Detailed explanation

Temperature rescales logits before probabilities are calculated. At low temperature, the distribution becomes sharper. At high temperature, lower-probability candidates gain more chance.

Temperature does not add knowledge and does not guarantee correctness. Its effect depends on the model and other decoding settings.

QA / AI quality perspective

Use low, stable settings for extraction, classification, and regulated answers. Test multiple seeds or repetitions for creative tasks. Record temperature in all defect evidence.

Real-world example

A JSON extractor may use temperature near zero, while a brainstorming assistant may use a higher value.

Common mistakes to avoid

- Saying zero temperature is perfectly deterministic across all systems
- Using high temperature to fix knowledge gaps
- Not testing parameter interactions

Likely follow-up questions

- How does temperature affect hallucination?
- When should temperature be low?
- What is greedy decoding?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q38. What is Top-K sampling?

Interview-ready answer

Top-K sampling restricts token selection to the K most probable candidates at each generation step.

Detailed explanation

After calculating probabilities, all but the top K candidates are removed and the remaining distribution is renormalized. A small K narrows output; a larger K allows more variety. The same K behaves differently depending on how concentrated the original distribution is.

QA / AI quality perspective

Test whether the selected setting balances consistency and flexibility for the task. Avoid assuming one universal K is appropriate.

Real-world example

With K=5, the model samples only among the five highest-probability next tokens.

Common mistakes to avoid

- Confusing generation Top-K with retrieval top-k
- Assuming K directly equals creativity
- Ignoring temperature interaction

Likely follow-up questions

- What is Top-P?
- Can Top-K and Top-P be combined?
- How do you tune decoding?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q39. What is Top-P or nucleus sampling?

Interview-ready answer

Top-P sampling keeps the smallest set of tokens whose cumulative probability reaches a threshold P , then samples from that set.

Detailed explanation

Unlike Top-K, the candidate count changes dynamically. If one token is highly likely, the nucleus may be small; if probabilities are spread out, it may contain more tokens. This adapts to uncertainty in each step.

QA / AI quality perspective

Evaluate consistency, quality, and safety across representative prompts. Log the setting and avoid changing multiple decoding parameters at once during experiments.

Real-world example

At $P=0.9$, candidates are included in descending probability order until their combined probability reaches roughly 90%.

Common mistakes to avoid

- Treating P as probability of correctness
- Confusing Top-P with confidence threshold
- Changing Top-P without regression tests

Likely follow-up questions

- Top-K vs Top-P?
- What is nucleus sampling?
- When is greedy decoding better?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q40. What is greedy decoding?

Interview-ready answer

Greedy decoding selects the highest-probability token at every step.

Detailed explanation

It is simple and relatively consistent but can produce repetitive or locally optimal text. It does not search for the globally best sequence. Some APIs approximate greedy behavior with very low temperature, though provider implementation details vary.

QA / AI quality perspective

Use it for deterministic-style tasks only after testing accuracy and repetition. Do not assume predictable output is correct output.

Real-world example

A structured field extractor selects the most likely next token at each step to minimize variation.

Common mistakes to avoid

- Equating greedy with globally optimal
- Using greedy decoding for all tasks
- Ignoring tie-breaking and backend variation

Likely follow-up questions

- What is beam search?
- Why can greedy output repeat?
- How do you evaluate determinism?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q41. How do Temperature, Top-K, and Top-P interact?

Interview-ready answer

Temperature reshapes probabilities, while Top-K and Top-P filter the candidate set; together they determine how constrained or diverse sampling is.

Detailed explanation

Using all three can make behavior hard to reason about. A common practice is to tune temperature and either top-k or top-p while leaving other values at defaults. Provider defaults and implementation vary, so experiments must be model-specific.

QA / AI quality perspective

Create controlled A/B evaluations with a fixed prompt dataset. Measure quality, safety, consistency, latency, and cost. Change one parameter at a time.

Real-world example

A higher temperature may have little visible effect if Top-K is set to 1 because only one token remains.

Common mistakes to avoid

- Tuning on one prompt
- Changing several controls simultaneously
- Copying settings between different models

Likely follow-up questions

- How do you design a decoding experiment?
- Which settings fit extraction?
- How do settings affect reproducibility?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q42. How would you test decoding parameters?

Interview-ready answer

Run a representative prompt dataset repeatedly across parameter configurations and compare correctness, relevance, safety, diversity, consistency, latency, and cost.

Detailed explanation

The test matrix should include deterministic tasks, open-ended tasks, adversarial prompts, and edge cases. Use fixed model versions and log parameters. For stochastic settings, use multiple trials and summarize distributions rather than one response. Select settings based on business quality objectives.

QA / AI quality perspective

QA should publish a parameter recommendation with evidence and quality thresholds, not merely a subjective preference.

Real-world example

For support answers, compare temperature 0, 0.2, and 0.7 over 200 prompts and measure unsupported-claim rate and response consistency.

Common mistakes to avoid

- Selecting by “sounds better”
- Using only one run per prompt
- Ignoring model updates

Likely follow-up questions

- What sample size is enough?
- How can response diversity be measured?
- How do you regression-test stochastic outputs?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Chapter 8

Context Windows and Memory

Simplified LLM inference flow

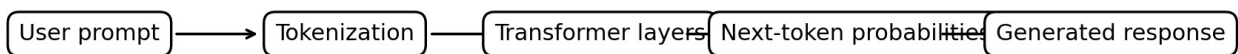


Figure: Context Windows and Memory - conceptual overview

This chapter contains 6 interview questions. Focus on the short answer first, then connect the concept to practical QA decisions and test design.

Q43. What is a context window?

Interview-ready answer

A context window is the maximum amount of tokenized input and generated output that a model can consider in one interaction.

Detailed explanation

It may include system instructions, conversation history, user input, retrieved documents, tool descriptions, tool results, and output tokens. A larger advertised window does not guarantee equal attention to every position. Long contexts increase cost and latency and can introduce distraction or conflicting evidence.

QA / AI quality perspective

Test near the limit, evidence at beginning/middle/end, conflicting passages, irrelevant content, truncation, and long multi-turn conversations. Validate the application's context assembly logic.

Real-world example

A 100-page document may fit in the nominal window but the model can still miss a critical clause buried in the middle.

Common mistakes to avoid

- Equating fit with reliable comprehension
- Ignoring hidden prompt tokens
- Not reserving space for output

Likely follow-up questions

- What is lost-in-the-middle?
- How do you manage long context?
- How does RAG help?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q44. What happens when input exceeds the context window?

Interview-ready answer

The request may be rejected, truncated, summarized, or otherwise reduced by the application or provider, potentially losing important information.

Detailed explanation

Behavior depends on API and application logic. Silent truncation is especially risky because instructions or evidence may disappear. Systems should apply explicit token budgeting, document selection, summarization, sliding windows, or RAG.

QA / AI quality perspective

Test oversized inputs and verify user messaging, logs, prioritization rules, and absence of unsafe silent behavior. Confirm that system instructions are never removed before lower-priority content.

Real-world example

A long conversation causes early customer consent details to be dropped, leading the assistant to repeat a restricted action.

Common mistakes to avoid

- Assuming the provider always returns an error
- Dropping content without observability
- Truncating from the wrong side

Likely follow-up questions

- What content should be prioritized?
- How do you reserve output tokens?
- What is context compression?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q45. What is the lost-in-the-middle problem?

Interview-ready answer

It is the tendency of some models to use information at the beginning or end of a long context more reliably than information located in the middle.

Detailed explanation

Attention and training patterns can cause position-dependent retrieval from long prompts. This is a behavioral limitation, not merely a token-limit issue. Reordering evidence, using retrieval, summarizing, or adding explicit references can help.

QA / AI quality perspective

Build positional robustness tests by placing the same critical fact at multiple locations while keeping the rest constant. Measure answer accuracy by position.

Real-world example

A model answers correctly when an exception clause is first or last but misses it when it appears halfway through a long policy.

Common mistakes to avoid

- Testing only one document order
- Assuming large context removes retrieval needs
- Not distinguishing position failure from missing knowledge

Likely follow-up questions

- How can it be mitigated?
- What is long-context evaluation?
- Should documents be reordered?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q46. How is conversation memory different from model memory?

Interview-ready answer

Conversation memory is context supplied by the application, while model memory refers to patterns encoded in model parameters during training.

Detailed explanation

An LLM API is typically stateless unless the platform stores history. Applications implement short-term history, summaries, vector memory, or user profiles. Persisted memory creates privacy, staleness, and contamination risks. The base model does not automatically remember a user across independent requests.

QA / AI quality perspective

Test session isolation, deletion, consent, stale memory, contradiction resolution, summarization loss, and cross-user leakage.

Real-world example

A support bot stores a customer's preferred language in an application database and inserts it into later prompts.

Common mistakes to avoid

- Saying the model remembers all conversations
- Ignoring privacy controls
- Confusing RAG knowledge with user memory

Likely follow-up questions

- What is short-term memory?
- What is vector memory?
- How do you test memory isolation?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q47. What is context engineering?

Interview-ready answer

Context engineering is the design of what information, instructions, tools, memory, and retrieved evidence are provided to a model for a task.

Detailed explanation

It goes beyond writing one prompt. It includes selection, ordering, formatting, access control, token budgeting, citations, tool schemas, examples, and conversation state. Good context engineering often improves quality more safely than fine-tuning.

QA / AI quality perspective

Test context assembly as a first-class component. Validate that only authorized, current, relevant, and correctly formatted information is included.

Real-world example

A support agent receives the active product version, relevant ticket history, approved troubleshooting steps, and a strict response schema.

Common mistakes to avoid

- Treating context as a text dump
- Ignoring precedence and ordering
- Including sensitive data unnecessarily

Likely follow-up questions

- Prompt engineering vs context engineering?
- What is context poisoning?
- How do you test context selection?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q48. How can long conversations be managed?

Interview-ready answer

Use techniques such as rolling windows, summarization, retrieval of relevant history, structured state, and explicit memory policies.

Detailed explanation

Naively appending every turn increases cost and distracts the model. Summaries reduce length but can lose details. Retrieval can surface relevant prior facts but may return stale or unrelated content. Structured state is suitable for critical fields. The best design often combines methods.

QA / AI quality perspective

Test summary accuracy, retained constraints, contradiction handling, user corrections, and privacy deletion. Compare behavior after many turns.

Real-world example

A travel agent stores dates and budget as structured state, summarizes discussion, and retrieves only relevant past preferences.

Common mistakes to avoid

- Keeping unlimited raw history
- Trusting summaries without validation
- Failing to update corrected facts

Likely follow-up questions

- How do you test memory updates?
- What is state management?
- When should a conversation reset?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Chapter 9

Fine-Tuning and Adaptation

Fine-tuning lifecycle



Figure: Fine-Tuning and Adaptation - conceptual overview

This chapter contains 6 interview questions. Focus on the short answer first, then connect the concept to practical QA decisions and test design.

Q49. What is fine-tuning?

Interview-ready answer

Fine-tuning further trains a pretrained model on task- or domain-specific examples to adapt its behavior or output style.

Detailed explanation

It changes model parameters or lightweight adapters. Fine-tuning can improve consistent formatting, domain language, classification behavior, or specialized tasks. It is not usually the best way to inject frequently changing factual knowledge; RAG is often better for that. Fine-tuning requires high-quality datasets, evaluation, safety checks, and monitoring.

QA / AI quality perspective

Validate dataset quality, baseline performance, overfitting, regressions on general capabilities, safety, and performance after training. Keep fixed holdout sets.

Real-world example

A company fine-tunes a model to classify support tickets into its internal taxonomy with consistent JSON output.

Common mistakes to avoid

- Using fine-tuning for current policies
- Training on unclean production logs
- Skipping baseline comparison

Likely follow-up questions

- RAG vs fine-tuning?
- What is instruction fine-tuning?
- How do you test overfitting?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q50. When should you use prompt engineering, RAG, or fine-tuning?

Interview-ready answer

Use prompting for behavior achievable through instructions, RAG for external or changing knowledge, and fine-tuning for persistent behavior or domain adaptation that prompting alone cannot reliably deliver.

Detailed explanation

The choice should follow measured gaps. Start with the simplest approach, establish a baseline, and escalate only when evidence supports it. Combinations are common: a fine-tuned model may still use RAG and strong prompts.

QA / AI quality perspective

QA should demand comparative evaluation across alternatives using the same test dataset and business metrics.

Real-world example

A policy assistant uses RAG for current documents, while a fine-tuned classifier routes requests and a prompt controls answer format.

Common mistakes to avoid

- Choosing the most complex technique first
- Using fine-tuning as a knowledge database
- Comparing approaches on different datasets

Likely follow-up questions

- What is a baseline?
- Can RAG and fine-tuning be combined?
- How do you calculate ROI?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q51. What is parameter-efficient fine-tuning (PEFT)?

Interview-ready answer

PEFT adapts a model by training a small subset or added set of parameters instead of updating the full model.

Detailed explanation

Methods such as LoRA add low-rank matrices to selected layers. PEFT reduces compute, storage, and deployment cost and allows multiple task adapters. Quality still depends on data and configuration. It can also preserve the base model while switching adapters.

QA / AI quality perspective

Test adapter selection, compatibility with base-model versions, task quality, cross-task leakage, latency, and rollback.

Real-world example

A company maintains separate adapters for legal drafting and customer-support tone on one base model.

Common mistakes to avoid

- Assuming PEFT cannot overfit
- Mixing an adapter with an incompatible base version
- Not testing adapter switching

Likely follow-up questions

- What is LoRA?
- What is QLoRA?
- How do adapters affect deployment?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q52. What is LoRA?

Interview-ready answer

LoRA is a PEFT method that learns low-rank updates to selected weight matrices while keeping the original model weights frozen.

Detailed explanation

Instead of storing a full copy of the model, LoRA stores small adapter parameters. This makes experimentation and deployment more efficient. Rank, target modules, learning rate, and dataset quality affect results. LoRA changes behavior but does not inherently solve factuality or safety.

QA / AI quality perspective

Benchmark the adapter against the base model and prompt-only baseline. Test domain gains, general regressions, safety, and reproducibility.

Real-world example

A LoRA adapter teaches a model to produce company-specific incident summaries without retraining the complete model.

Common mistakes to avoid

- Calling LoRA a new model architecture
- Ignoring base-model dependency
- Evaluating only training loss

Likely follow-up questions

- How is LoRA deployed?
- What is adapter merging?
- What metrics evaluate a fine-tuned model?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q53. How do you test a fine-tuned LLM?

Interview-ready answer

Compare it with a fixed baseline using held-out datasets for target quality, safety, robustness, general regressions, latency, and cost.

Detailed explanation

The strategy should cover data leakage, contamination, memorization, overfitting, class imbalance, format adherence, harmful behavior, and drift. Use both automated metrics and human evaluation. Maintain model cards and experiment records.

QA / AI quality perspective

Create immutable benchmark sets and acceptance thresholds before training. Run back-to-back tests between base and tuned versions and investigate regressions by category.

Real-world example

A tuned support model improves ticket routing from 82% to 91% but starts misclassifying rare security incidents, blocking release.

Common mistakes to avoid

- Testing on training data
- Accepting aggregate improvement while a critical class degrades
- Skipping safety regression

Likely follow-up questions

- What is data contamination?
- What is back-to-back testing?
- How do you detect memorization?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q54. What are risks of fine-tuning?

Interview-ready answer

Risks include overfitting, catastrophic forgetting, bias amplification, memorization, privacy leakage, unsafe behavior, and maintenance complexity.

Detailed explanation

A narrow dataset can distort behavior. Sensitive examples may be reproduced. Poor labels can teach the wrong policy. Model-provider updates can break adapter compatibility. Fine-tuning also creates governance obligations for dataset provenance, approval, evaluation, and rollback.

QA / AI quality perspective

Use data governance, deduplication, privacy review, holdout sets, safety testing, versioning, and rollback plans. Red-team the tuned model independently.

Real-world example

A model fine-tuned on past support responses learns an outdated workaround and repeatedly suggests it after the product changes.

Common mistakes to avoid

- Assuming more domain data is always better
- Ignoring license and consent
- Not planning model retirement

Likely follow-up questions

- What is catastrophic forgetting?
- How can privacy leakage be tested?
- What should a model card contain?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Chapter 10

Prompt Engineering Basics

Anatomy of a strong prompt



Figure: Prompt Engineering Basics - conceptual overview

This chapter contains 6 interview questions. Focus on the short answer first, then connect the concept to practical QA decisions and test design.

Q55. What is prompt engineering?

Interview-ready answer

Prompt engineering is the systematic design and evaluation of instructions, context, examples, and output constraints used to guide a model.

Detailed explanation

It includes role definition, task clarity, context selection, constraints, examples, tool instructions, and output schemas. Effective prompting is an engineering discipline because prompts must be versioned, tested, observed, and improved against measurable requirements.

QA / AI quality perspective

Treat prompts as code. Store them in version control, create regression datasets, compare versions, and define quality gates for correctness, safety, and format.

Real-world example

A defect summarizer prompt specifies audience, severity rules, required fields, prohibited assumptions, and JSON schema.

Common mistakes to avoid

- Using vague natural language without acceptance criteria
- Changing prompts directly in production
- Testing only one example

Likely follow-up questions

- What is a system prompt?
- What is prompt versioning?
- How do you regression-test prompts?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q56. What is a system prompt?

Interview-ready answer

A system prompt is a high-priority instruction that defines the model's role, behavior, policies, and constraints for an interaction.

Detailed explanation

It often contains safety rules, domain boundaries, style, tool-use instructions, and output requirements. It is not an absolute security boundary; prompt injection or conflicting context can still influence behavior. Applications must combine it with authorization, validation, and guardrails.

QA / AI quality perspective

Test direct and indirect attempts to override it, multilingual attacks, encoded instructions, conflicting documents, and tool misuse. Never place secrets in prompts.

Real-world example

A banking assistant system prompt prohibits revealing account data and requires identity verification before tool calls.

Common mistakes to avoid

- Treating the system prompt as unbreakable
- Embedding credentials
- Relying on prompt rules instead of access control

Likely follow-up questions

- What is prompt injection?
- How are instructions prioritized?
- What are guardrails?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q57. What is zero-shot prompting?

Interview-ready answer

Zero-shot prompting asks the model to perform a task using instructions without providing task-specific examples.

Detailed explanation

It is fast and simple, and works well when the task is common and the model understands the requested format. Performance may suffer for specialized labels, subtle rules, or unusual output structures. It is a useful baseline before adding examples or fine-tuning.

QA / AI quality perspective

Create a baseline evaluation set and measure output quality before deciding that examples are necessary. Test instruction paraphrases and edge cases.

Real-world example

“Classify this defect as Critical, Major, Minor, or Trivial using the policy below” without sample classifications.

Common mistakes to avoid

- Assuming zero-shot means no context
- Adding examples before measuring a baseline
- Using undefined labels

Likely follow-up questions

- What is few-shot prompting?
- When is zero-shot sufficient?
- How can a rubric improve zero-shot output?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q58. What is few-shot prompting?

Interview-ready answer

Few-shot prompting includes a small set of input-output examples to demonstrate the desired behavior or format.

Detailed explanation

Examples can clarify labels, tone, reasoning style, and edge-case handling. They consume context and can bias the model toward the shown patterns. Example quality and order matter, and examples must not contain sensitive data.

QA / AI quality perspective

Test with different example orders, unseen edge cases, and adversarial cases. Verify that the model generalizes rather than copying irrelevant details.

Real-world example

Provide three sample bug reports with correct severity and rationale before asking the model to classify a new one.

Common mistakes to avoid

- Using inconsistent examples
- Including only easy cases
- Assuming more examples always improve quality

Likely follow-up questions

- How do you select examples?
- What is example contamination?
- What is in-context learning?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q59. What is chain-of-thought prompting?

Interview-ready answer

Chain-of-thought prompting encourages intermediate reasoning steps for tasks that benefit from decomposition, although production systems often keep hidden reasoning private and request concise justifications instead.

Detailed explanation

Explicit step-by-step reasoning can improve some tasks but can also add cost, latency, and unreliable explanations. Modern best practice is to ask for structured evidence, calculations, or concise rationale rather than exposing or depending on hidden internal reasoning.

QA / AI quality perspective

Validate final outputs and externally checkable evidence. Do not treat a fluent rationale as proof. Test whether the same result is reached with different formulations.

Real-world example

For a policy decision, request the applicable rule IDs and final decision rather than unrestricted internal monologue.

Common mistakes to avoid

- Equating explanation with true reasoning
- Storing sensitive reasoning logs unnecessarily
- Using verbose reasoning for simple tasks

Likely follow-up questions

- What is self-consistency?
- What is structured reasoning?
- How do you verify reasoning?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Q60. How do you test prompts?

Interview-ready answer

Use versioned prompt datasets, deterministic checks, semantic evaluation, safety tests, repeated runs, human review, and A/B comparisons.

Detailed explanation

A prompt test suite should cover normal, boundary, ambiguous, adversarial, multilingual, long-context, and unanswerable inputs. Assertions may include schema validity, required fields, groundedness, refusal behavior, tone, latency, and cost. Changes should run automatically in CI where practical.

QA / AI quality perspective

Define a prompt contract and quality thresholds. Record model and parameter versions. Review category-level regressions rather than only aggregate score.

Real-world example

A new support prompt improves helpfulness but increases unsupported claims on refund questions, so the release is blocked.

Common mistakes to avoid

- Testing only the prompt text without the full system context
- Using exact string assertions for all outputs
- Approving on average score while critical cases fail

Likely follow-up questions

- What is prompt regression?
- What is an evaluation dataset?
- How do you define prompt quality gates?

Interview tip: Start with the concise definition, add one practical example, and then explain how you would validate the behavior as a QA professional.

Quick Revision Checklist

- ☐ Explain AI, ML, Deep Learning, and Generative AI without mixing the terms.
- ☐ Describe the complete LLM inference flow from prompt to next-token generation.
- ☐ Explain self-attention, multi-head attention, and positional information at a practical level.
- ☐ Explain tokens, tokenization, context budgets, and cost implications.
- ☐ Describe embeddings, semantic similarity, and vector retrieval.
- ☐ Differentiate vector databases, vector indexes, metadata filters, and ANN search.
- ☐ Compare temperature, Top-K, Top-P, and greedy decoding.
- ☐ Explain context windows, lost-in-the-middle, conversation memory, and context engineering.
- ☐ Choose between prompt engineering, RAG, and fine-tuning using business evidence.
- ☐ Design a basic prompt test strategy with versioning, datasets, metrics, and quality gates.

Glossary

Term	Meaning
AI	Systems designed to perform tasks associated with human intelligence.
ANN	Approximate Nearest Neighbor search for efficient vector retrieval.
Context Window	Maximum token budget a model can process in one interaction.
Embedding	Numeric vector representation of semantic content.
Fine-Tuning	Additional training that adapts a pretrained model.
Foundation Model	General-purpose model reusable across many tasks.
Hallucination	Plausible but unsupported or incorrect generated content.
LLM	Large Language Model trained to model and generate token sequences.
LoRA	Parameter-efficient adaptation using low-rank weight updates.
RAG	Retrieval-Augmented Generation; external evidence is retrieved and supplied to a model.
Self-Attention	Mechanism that relates tokens to other tokens in the same sequence.
Token	Text unit processed by a model.
Top-K	Sampling restricted to the K most probable next tokens.
Top-P	Sampling restricted to the smallest token set reaching cumulative probability P.
Vector Database	Storage and search system optimized for embeddings.

Selected References

- Vaswani et al. - Attention Is All You Need (2017).
- Official documentation of major model providers for tokenization, context limits, embeddings, and generation parameters.
- Lewis et al. - Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (2020).
- ISTQB materials on AI testing and Generative AI testing for quality-engineering context.
- NIST AI Risk Management Framework for responsible AI and risk terminology.
- OWASP guidance for LLM application security and prompt-injection awareness.

About the Author

Mohit Gupta is a Quality Assurance Lead with more than twelve years of experience in enterprise software testing, test strategy, automation, API validation, and quality engineering. He is pursuing advanced study in Artificial Intelligence and Machine Learning and is building expertise in AI testing, LLM evaluation, RAG validation, and AI quality architecture.